Proiectare Logică

- I CB: Mariana Mocanu mariana.mocanu@cs.pub.ro
- **I CC: Anca Morar** anca.morar@cs.pub.ro
- I CA & CD: Costin Chiru costin.chiru@cs.pub.ro

Curs 13: Structuri microprogramate: microinstructiuni cu format variabil

Recapitulare (1)

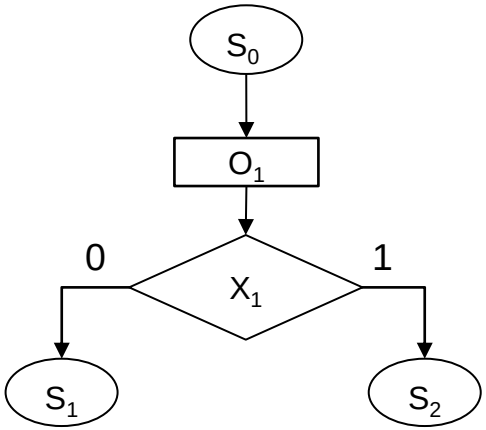
- De-a lungul semestrului ați învățat să faceți sinteza circuitelor folosind CBB-uri și identificând ecuațiile de stare și ecuațiile de intrare/ieșire din CBB-uri
  - Problemă: se poate aplica această metodă pentru un număr relativ mic de stări (altfel diagramele Karnaugh devin prea mari)
- Am făcut cursul trecut o altă metodă de sinteză a circuitelor folosind structuri microprogramate
- Ideea de bază e că avem o succesiune de stări și tranziții (condiționate de variabile de intrare sau necondiționate) ce pot fi salvate în structurile microprogramate

Recapitulare (2)

- În practică, automatele cu multe stări sunt frecvente și nu putem să le „spargem” pentru că nu știm să lucrăm cu mai mult de 5 variabile → de aceea se folosesc structurile microprogramate
- Alt avantaj al structurilor microprogramate este fiabilitatea crescută
- Microinstrucțiunile sunt cuprinse în microprogram, acesta fiind salvat în memorie
- Data trecută am făcut microinstrucțiuni cu format fix și am spus că vom învăța o metodă de a elimina informația redundantă

Microinstrucțiuni cu format fix (1)

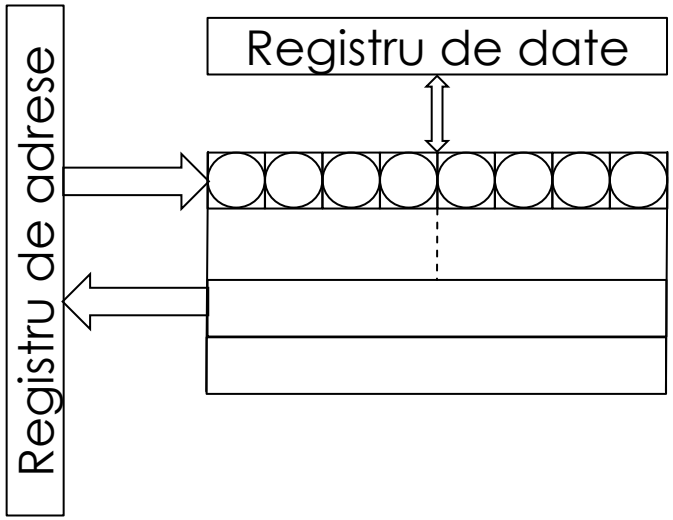
- Ce trebuie să conțină?
- Adresa stării curente, informații referitoare la condiție, unde pot merge mai departe și informații despre eventualele ieșiri



Input Code	ADR 0	ADR 1	OUT

- $L_{\mu i} = n_{inpcod} + 2 * n_{ADR} + n_{OUT}$

- RA selectează adresa, iar prin intermediul RD se scriu/citesc date în/din memorie



Microinstrucțiuni cu format fix (2)

Input Code	ADR 0	ADR 1	OUT



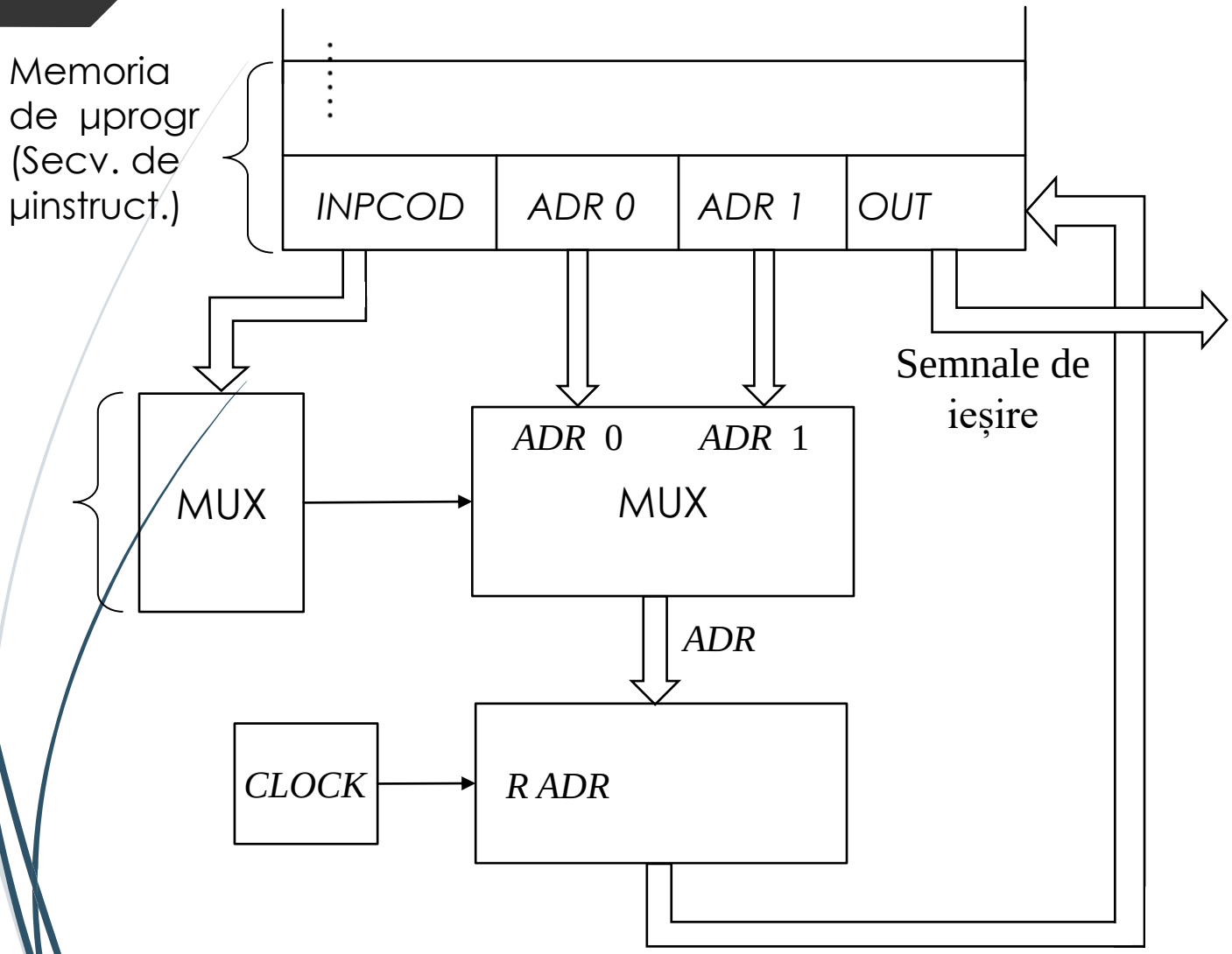
- Cele n intrări sunt codificate, iar codul lor e folosit în prima parte a microinstrucțiunii pentru a stabili care dintre intrări e responsabilă de tranziție
- ADR0, ADR1 – adresele pe 0/1 = locul unde se „sare" cu fluxul de control al aparatului
- OUT = ieșirile, nu se codifică → numărul de ieșiri = numărul de biți OUT necesari
- $L_{\mu i}$ este suma biților necesari pentru fiecare câmp în parte

Microinstrucțiuni cu format fix (3)

- Avem 2 situații posibile:

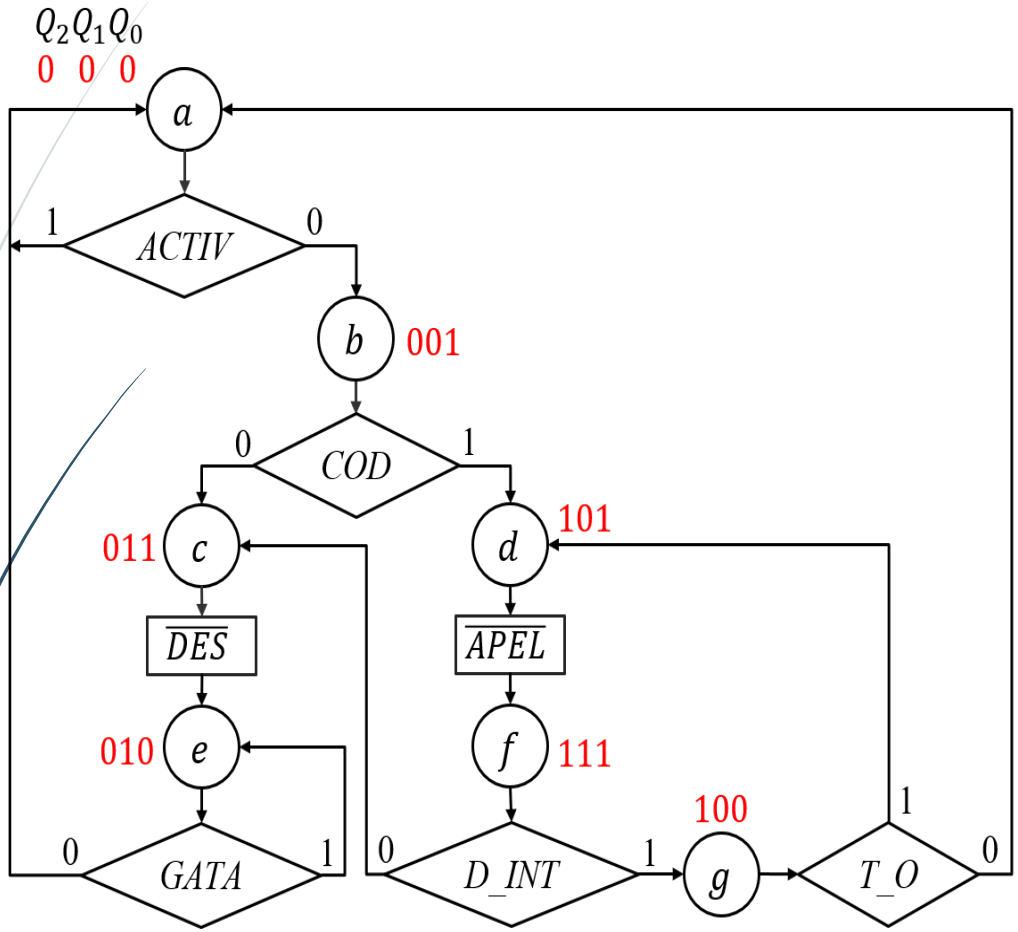
1. Când memoria este impusă și atunci pe baza acesteia determinăm numărul de biți pentru RA. Lungimea cuvântului de memorie poate fi oricât de lung și atunci folosim mai multe circuite de memorie în paralel.
   Ex: memorie de 1k = 2^{10} → RA are 10 biți
   Ex2: memorie de 8 cuvinte = 2^3 → RA are 3 biți
2. Pornind de la organigramă → numărul de stări → numărul de variabile de stare → numărul de biți al RA. Dacă sunt mai multe zone de memorie decât stări, atunci rămân zone nefolosite.
   (Nu este necesar să stabilim coduri pentru stări care să difere printr-un singur bit, ca la CBB-uri!)

Structura de comandă microprogramată



Observație: trebuie să știți să explicați structura de comandă microprogramată, dar nu e nevoie să o reproduceți → trebuie doar să știți să calculați $L_{\mu i}$ și să completați memoria de microprogram

Problema cu interfonul

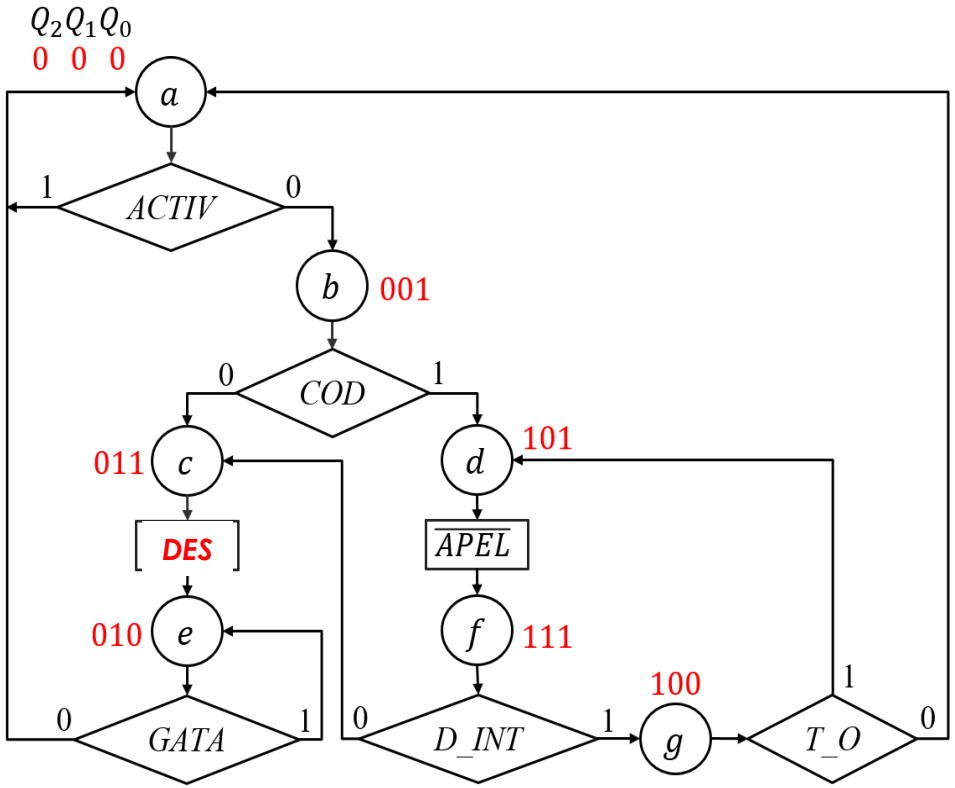


- Câte intrări?
- 5
- Câți biți?
- 3
- Care sunt intrările?
- ACTIV, COD, GATA, D_INT, T_O
- Câte stări?
- 7
- Câți biți?
- 3
- Câte ieșiri?
- 2 (!APEL, !DES)
- Cât e $L_{\mu i}$?

$L_{\mu i} = 3 + 2 * 3 + 2 = 11$ biți

Problema cu interfonul – memoria de microprogram (3)

ACTIV – 001, COD – 010, GATA – 011, D-INT – 100, T_0 -101



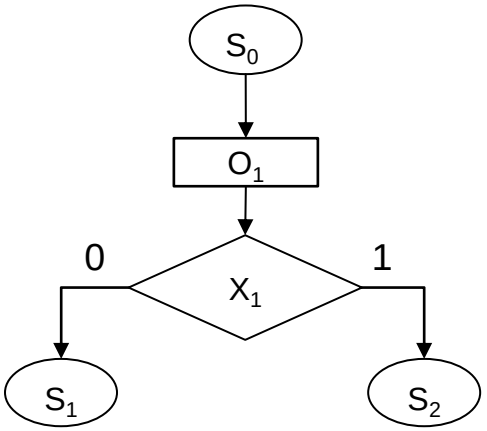
	INPCOD	ADR0	ADR1	DES	APEL
a - 000	001	001	000	0	1
b - 001	010	011	101	0	1
e - 010	011	000	010	0	1
c - 011	000	010	010	1	1
g - 100	101	000	101	0	1
d - 101	000	111	111	0	0
- - 110	xxx	xxx	xxx	x	x
f - 111	100	011	100	0	1

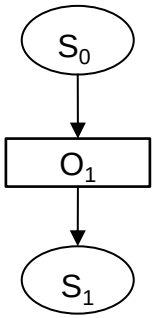
- Nu rămân mereu coduri nefolosite (intrări = $2^{\text{biți}}$), iar dacă nu rămân, punem un cod oarecare
- La salturile necondiționate se pune același lucru pe ADR0 și ADR1

- Ce se întâmplă în C?
- Putem pune XXX sau, dacă rămân coduri nefolosite, pun 000 → de asta începem codificarea de la 001
- 000 va fi codul pentru salturile necondiționate

Microinstrucțiuni cu format variabil (1)

- În cazurile semnalate se pierde spațiul de memorie fie prin specificarea de ieșiri inutile, fie prin dublarea adresei următoare → încercăm să optimizăm memoria folosită prin folosirea microinstrucțiunilor cu format variabil
- Există 2 tipuri de structuri în cazul microinstrucțiunilor cu format variabil:

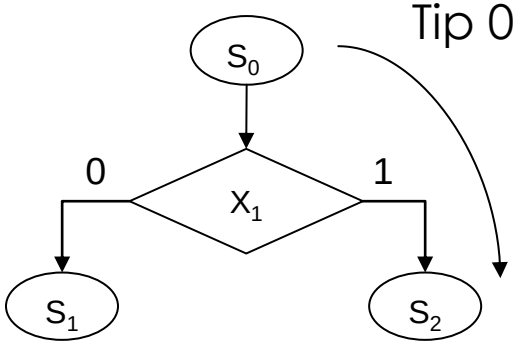




Tip 1

adr S1 = adr S0 +1

Se generează doar ieșiri
Saltul este necondiționat



Tip 0

adr S_2 = adr S_0 + 1

Doar decizie
Nu avem ieșiri

Microinstrucțiuni cu format variabil (2)

- Dacă avem o structură de Tip 1, avem nevoie doar de adresa următoare și de ieșiri, nu de 2 adrese + inpcod + ieșiri
- Dacă avem o structură de Tip 0, atunci avem nevoie de 2 adrese + inpcod, dar nu mai avem ieșiri
- În plus, dacă la RA de la memorie avem un numărător, atunci am putea avea adrese consecutive:
  - la Tip 1: adr S_1 = adr S_0 + 1 și ne interesează doar ieșirile,
  - la Tip 0: adr S_3 = adr S_1 + 1 și ne interesează doar intrarea (inpcod) și ADR_0
- Convenție: la microinstrucțiuni de Tip 0, când intrarea este 1, adresa următoare este adresa curentă + 1

Microinstrucțiuni cu format variabil (3)

- Având o astfel de memorie cu numărător la RA:
- $L_{\mu i\ Tip\ 1} = 1 + n_{out}$

1 | ieșiri

- 1 este pentru tipul instrucțiunii
- $L_{\mu i\ Tip\ 0} = 1 + n_{ci} + n_{adr0}$

0 | INPCOD | ADR0

- De cele mai multe ori microinstrucțiunile de Tip 1 și Tip 0 vor avea lungime diferită pentru că depind de logica programului (număr de stări, număr de ieșiri, număr de variabile de intrare)
- Pentru a determina $L_{\mu i}$ se va considera maximul dintre cele 2 valori:

$$L_{\mu i} = \max\left(L_{\mu i\ Tip\ 1}, L_{\mu i\ Tip\ 0}\right)$$

Microinstrucțiuni cu format variabil (4)

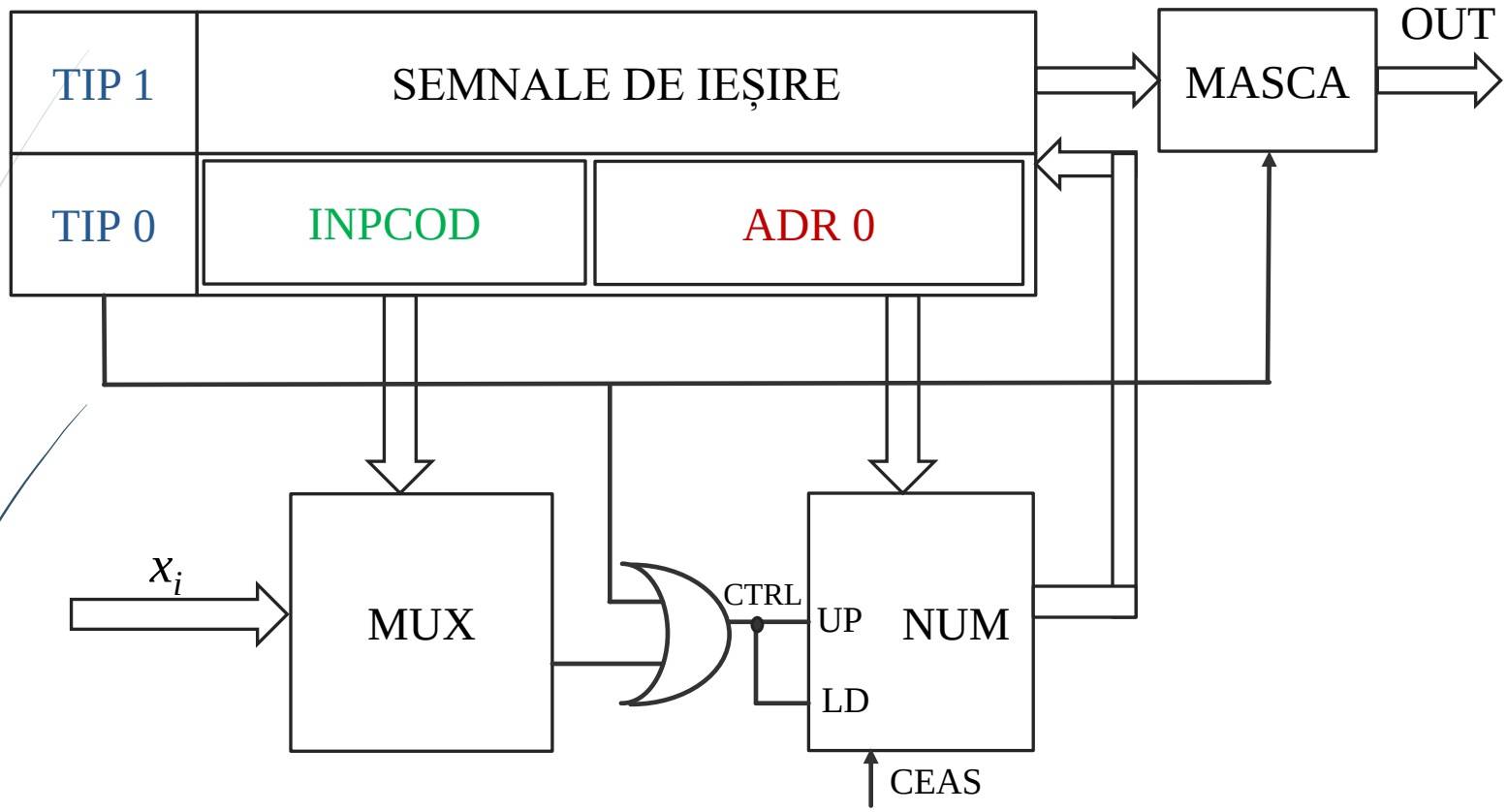
- Ex: adrese pe 4 biți, tot 4 biți pentru codul variabilelor de intrare și 30 ieșiri:
  - $L_{\mu i\ Tip\ 1} = 1 + 30 = 31$
  - $L_{\mu i\ Tip\ 0} = 1 + 4 + 4 = 9$
  - $L_{\mu i} = \max(L_{\mu i\ Tip\ 1}, L_{\mu i\ Tip\ 0}) = 31$

- Pentru asta:
  1. Memoria trebuie să știe să diferențieze microinstrucțiunile de tip 1 și 0 între ele (se modifică unitatea de comandă) și
  2. Trebuie modificată organigrama pentru a avea numai instrucțiuni de tip 1 și 0 prin adăugare de stări.

Structura de comandă microprogramată (1)

- Trebuie să ne asigurăm că putem trece de la adresa curentă la adresa incrementată, dar și că se poate face salt la o altă adresă dacă avem instrucțiune de tip 0 și intrarea este 0 → se folosește un numărător universal pentru că permite incrementarea, precum și presetarea (folosind LOAD) unei anumite valori și continuarea incrementării de la acel punct
- Acest numărător joacă de fapt rolul RA

Structura de comandă microprogramată (2)

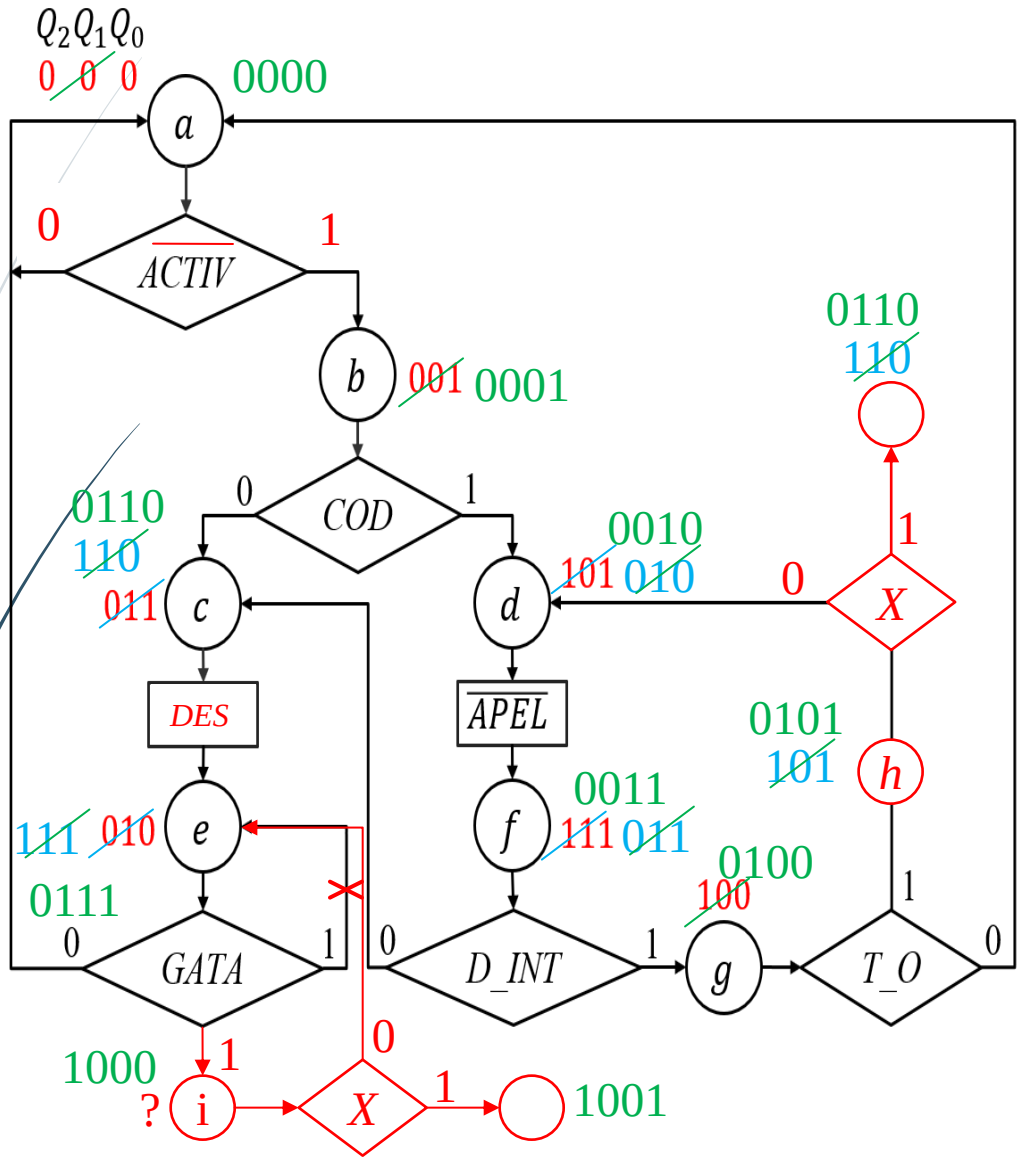


- Nu trebuie reprodusă la examen! Trebuie doar să o înțelegeți

Structura de comandă microprogramată (3)

- Dacă Tip = 1, Ctrl = 1
  - adresa curentă se incrementează (Up = 1)
  - se activează masca pentru a putea citi ieșirile (acestea se citesc direct)
- Dacă tip = 0
  - se dezactivează masca pentru a nu putea citi ieșirile
  - Dacă intrarea este = 1, Ctrl = 1 și adresa curentă se incrementează (Up = 1)
  - Dacă intrarea este = 0, Ctrl = 0, Load se activează și se încarcă ADR0 în numărător

Modificarea organigramei (1)



- Pas1: verificăm că avem doar instrucțiuni de tip 1 sau 0
- Pas 2: verificăm Tip 1: adresa următoare = adresa curentă + 1 și același lucru la Tip 0 când variabila de intrare = 1

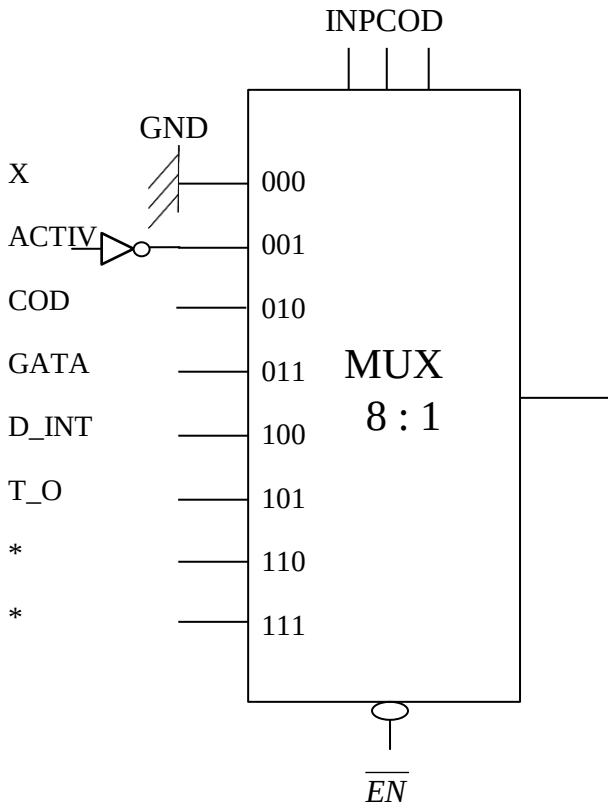
$X = 0$ → e posibil să se mărească numărul de biți pentru codificarea variabilelor de intrare

Modificarea organigramei (2)

- Din g oricum aș pune nu pot să fac tranziția pe 1 și atunci mai introduc o stare și o variabilă de intrare care va fi legată la masă (= 0 tot timpul). Astfel, folosesc din nou tranziția pe 0 ca să ajung unde doresc, iar tranziția pe 1 se duce către o stare unde nu ajung niciodată.
- Observații:
  - 1. codul stărilor unde nu se ajunge niciodată poate fi refolosit
  - 2. variabila de intrare legată la masă poate și ea fi refolosită de oricâte ori este nevoie pentru astfel de tranziții

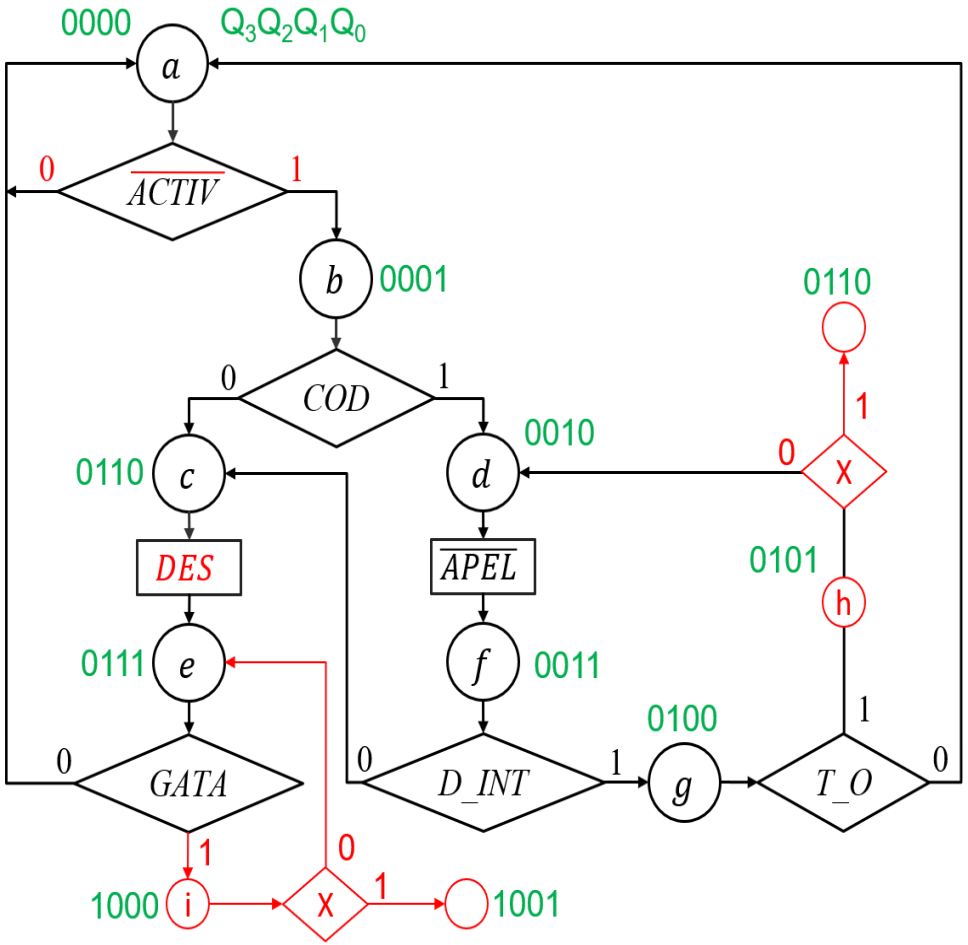
Modificarea organigramei (3)

- Ce înseamnă că modific ACTIV în !ACTIV?
- În unitatea de comandă, în MUX, în loc să pun ACTIV, pun !ACTIV
- !ACTIV va avea același cod ca ACTIV sau trebuie să neg biții?
- Are același cod și doar neg intrarea
- Pot să o fac mereu?
- Nu! Doar atunci când respectiva variabilă de intrare este folosită o singură dată (într-un singur loc).
- Altfel ce fac?
- Introduc o altă variabilă de intrare!



Memoria de microprogram

ACTIV – 001, COD – 010, GATA – 011, D-INT – 100, T_0 -101



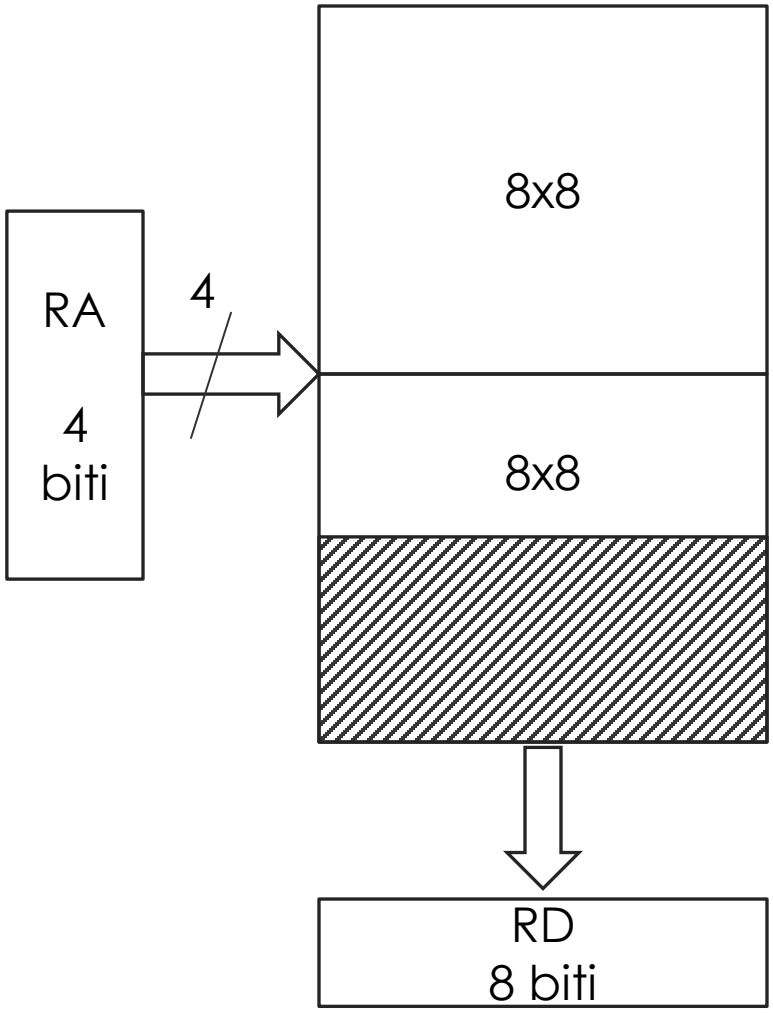
- $L_{\mu i\ tip\ 1} = 1 + 2 = 3$
- $L_{\mu i\ tip\ 0} = 1 + 3 + 4 = 8$
- $L_{\mu i} = \max(L_{\mu i\ tip\ 1}, L_{\mu i\ tip\ 0}) = 8$

Adr	*TIP* | *DES + APEL* (*COD* + *ADR0*)
a - 0000	0 | 001 0000
b - 0001	0 | 010 0110
d - 0010	1 | 00x xxxx
f - 0011	0 | 100 0110
g - 0100	0 | 101 0000
h - 0101	0 | 000 0010
c – 0110	1 | 11x xxxx
e - 0111	0 | 011 0000
i - 1000	0 | 000 0111
* - 1001	x | xxx xxxx

Memoria de microprogram

- Având în vedere că de cele mai multe ori cele 2 tipuri de microinstrucțiuni au dimensiuni diferite, trebuie ca una dintre ele să se completeze cu x-uri. Acest lucru se poate face fie la MSB, fie la LSB – e la latitudinea voastră
- Memorie consumată:
  - Format fix: 7 stări *11 biți = 77 biți
  - Format variabil: 9 stări * 8 biți = 72 biți

Memorii 8 x 8 – pentru μi cu format variabil

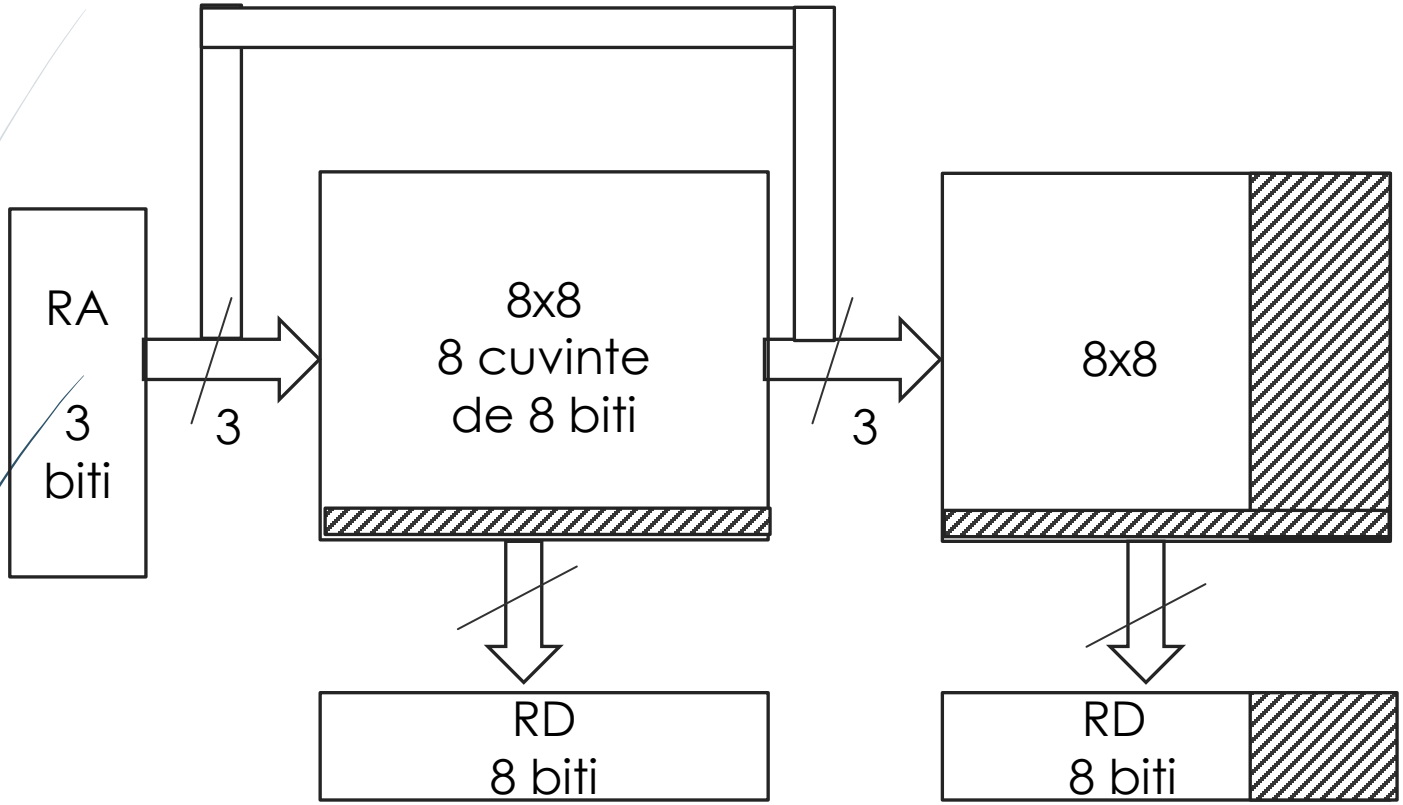


Memorii 8 x 8

μi cu format variabil:
11 stări x 8 biți

- 2 chip-uri de memorie

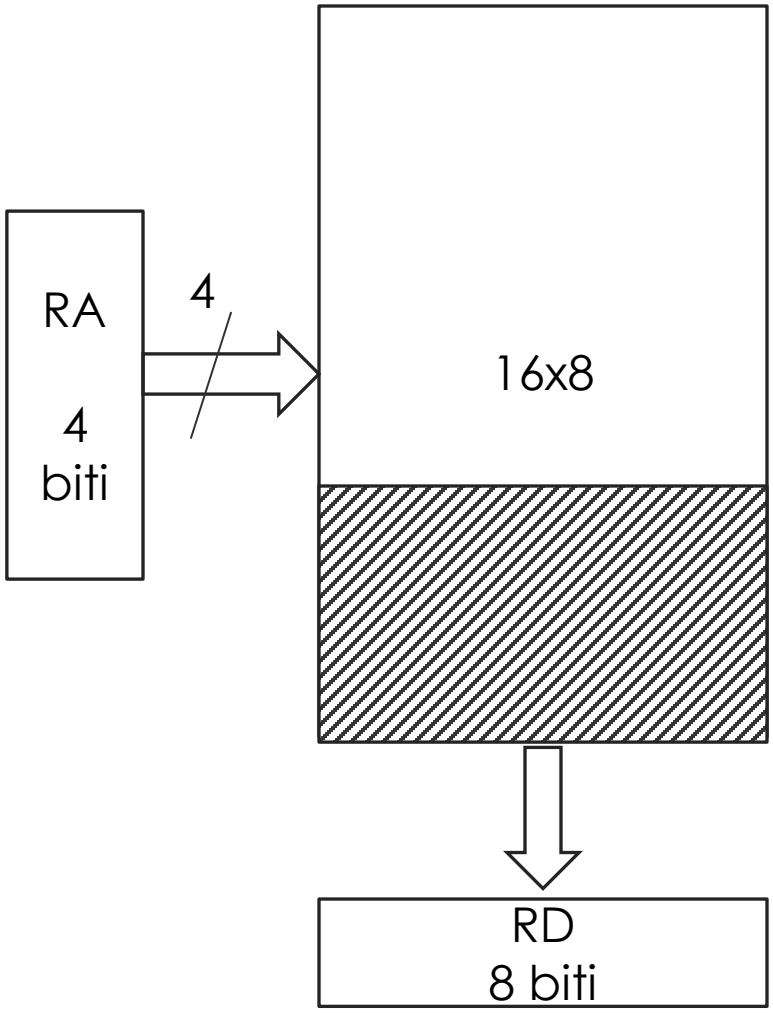
Memorii 8 x 8 – pentru μi cu format fix



Memorii 8 x 8

μi cu format fix: 7 stări * 14 biți
- 2 chip-uri de memorie

Memorii 16 x 8 – pentru μi cu format variabil

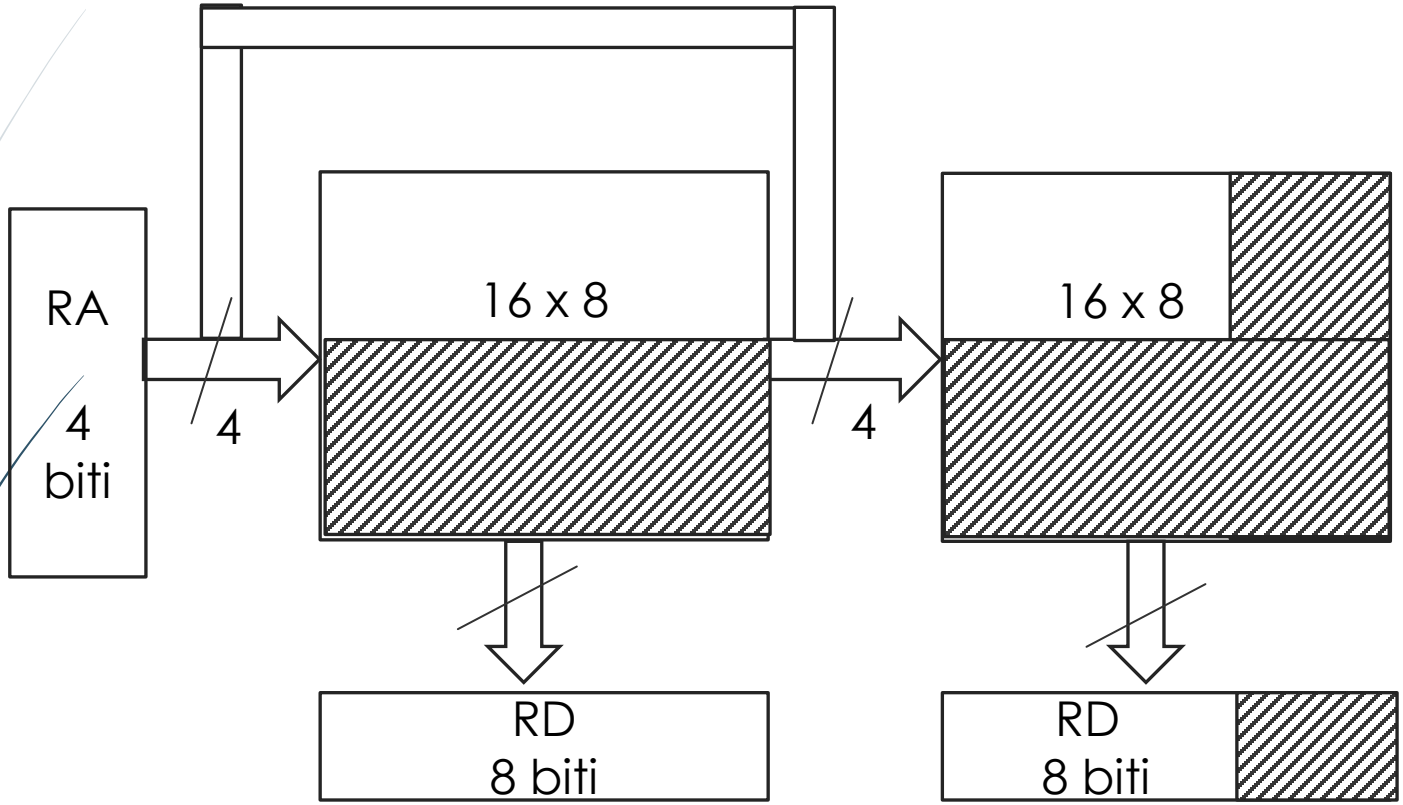


Memorii 16 x 8

μi cu format variabil:
9 stări x 8 biți

- 1 chip de memorie

Memorii 16 x 8 – pentru μi cu format fix



Memorii 16 x 8

μi cu format fix: 7 stări * 14 biți
- 2 chip-uri de memorie

Memorii 16 x 16

